개인 프로젝트

SKLabs

AI 주식 비서

실제 시세·뉴스 데이터를 모아 두고, 질문을 받으면
AI가 필요한 데이터를 직접 조회해서 답하는 서비스입니다.
개발부터 AWS 배포, 자동 배포까지 혼자 만들었습니다.

만든 사람	서원
서비스 주소	http://52.62.97.114
GitHub	github.com/SeoWon1037

주요 기능

AI 비서

종목·시세·수익률·뉴스를 도구로 조회해서 답변

주식 화면

12개 종목 목록, 캔들 차트, 일별 시세, 종목 뉴스

메인 화면

분야별 뉴스, 날씨, AI 검색창

회원 · 관리자

이메일 인증 가입, 카카오·네이버 로그인, 계정 찾기,
피드백 관리

이 프로젝트를 만든 이유

회사에서는 오래된 스택으로 ERP를 운영하고 있어서, 개인 개발을 시작하였습니다.

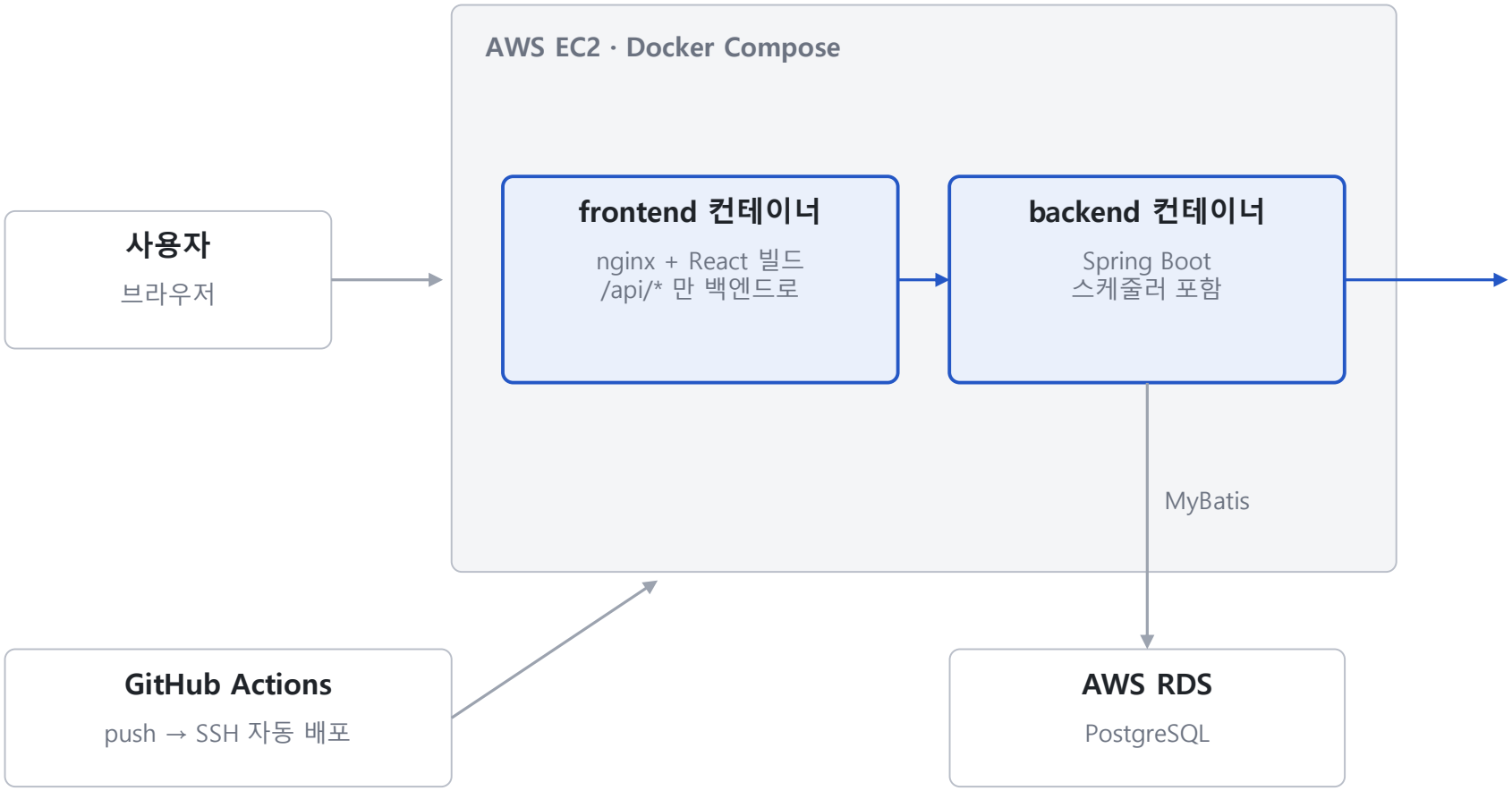
구분	회사 (사내 ERP · 연동 서버)	개인 프로젝트 (SKLabs)
Java	Java 1.6 / 1.8	Java 17
프레임워크	Spring 2.5, Spring Boot 2.2 / 2.3	Spring Boot 4.1
데이터 접근	iBATIS 2.3, MyBatis	MyBatis
DB	Oracle 11g (사내 서버)	PostgreSQL (AWS RDS)
화면	MiPlatform 3.2, JSP	React (Vite)
서버 · 배포	WebLogic, SVN, Jenkins	Docker, AWS EC2, GitHub Actions
AI	-	Claude API (tool use)

개발 환경과 사용 기술

영역	사용 기술	쓴 곳
백엔드	Java 17, Spring Boot 4.1, MyBatis, RestClient, Jackson	API, 스케줄러, 외부 API 호출
DB	PostgreSQL (AWS RDS, 시드니 리전)	회원, 시세, 뉴스, AI 대화 기록
프론트	React (Vite), react-router, lightweight-charts	메인, 주식, AI 비서, 관리자 화면
AI	Claude Messages API (tool use)	AI 주식 비서
외부 데이터	토스증권 API, Google 뉴스 RSS, Open-Meteo	시세, 뉴스, 날씨
인증	이메일 인증(SMTP), BCrypt, 카카오·네이버 OAuth 2.0	회원가입, 로그인, 계정 찾기
배포	Docker (멀티 스테이지), Docker Compose, nginx, AWS EC2	운영 서버
CI/CD	GitHub Actions	빌드·테스트, push 시 자동 배포
개발 도구	IntelliJ, pgAdmin, Git·GitHub, AI 코딩 에이전트(Claude)	개발, DB 확인, 형상 관리

전체 구성

EC2 한 대에 Docker Compose로 nginx와 Spring Boot를 띄우고, DB는 RDS를 씁니다. 외부에는 80 포트만 열었습니다.



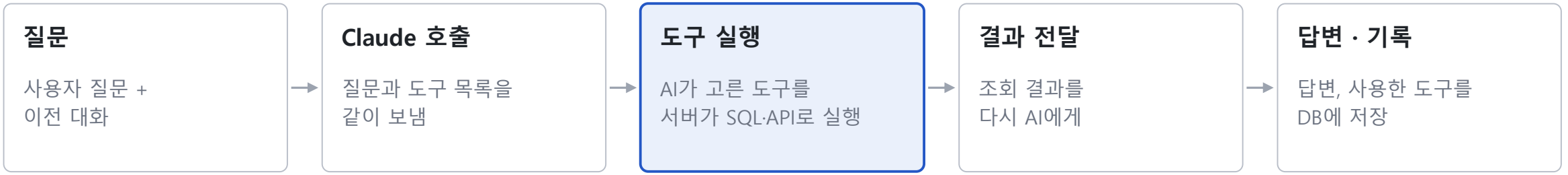
백엔드가 호출하는 외부 서비스

Claude API	AI 답변
토스증권 API	시세
Google 뉴스 RSS	뉴스
카카오 · 네이버	OAuth
네이버 메일	인증 메일

날씨는 브라우저에서 Open-Meteo를 바로 호출합니다.

LLM API 활용: AI가 데이터를 직접 조회

AI가 숫자를 지어내지 않도록, 필요한 데이터는 도구로 조회하게 만들었습니다. 어떤 도구를 쓸지는 AI가 고르고 서버는 실행만 합니다.



도구	하는 일	구현
find_stock	종목명으로 종목 찾기	SQL
get_current_price	현재가 조회	토스증권 API
get_price_history	최근 N일 시세	SQL
get_return_ranking	N거래일 수익률 순위	SQL 윈도우 함수
search_stock_docs	종목 설명 문서 검색 (RAG)	SQL 키워드 검색
search_news	수집해 둔 뉴스 검색	SQL

안전 장치

- 로그인 사용자만, 질문 500자 제한
- 도구 반복 최대 5회
- "숫자는 도구로 조회한 값만" 프롬프트
- 투자 권유 대신 데이터 설명만
- 화면에 사용한 도구를 같이 표시

답변을 검증하고, 기록을 보고 고쳤습니다

질문·답변·사용한 도구를 전부 DB에 남기고 👍/👎 평가를 받습니다. 관리자 화면에서 이 기록을 보고 개선할 점을 찾았습니다.

1 수익률 순위를 AI가 직접 계산하고 있었습니다

문제 "최근 20일 수익률 1등은?" 질문에 시세 조회를 종목마다 해서 도구를 6번 호출했고, AI가 직접 계산하다 반올림 오차가 났습니다. (-12.67%, 실제 -12.68%)

해결 계산은 SQL(윈도우 함수)이 하는 get_return_ranking 도구를 만들고, AI는 결과를 설명만 하게 했습니다. 도구 6번 → 1번, 값도 정확해졌습니다.

2 긴 답변이 중간에 끊겼습니다

문제 여러 종목을 비교하는 질문에서 답변 길이 제한에 걸려 문장이 잘린 채로 나갔습니다.

해결 max_tokens를 2048로 늘리고, 응답이 길이 제한(stop_reason: max_tokens)으로 끝난 경우를 따로 처리했습니다. 프롬프트에 핵심부터 짧게 요약하라는 규칙도 넣었습니다.

정확한 계산은 SQL과 코드에 맡기고, AI는 질문 이해·도구 선택·설명을 맡기는 쪽이 정확도와 비용 모두 나았습니다.

수집 → 변환 → 적재

데이터 파이프라인

AI와 화면이 쓰는 데이터는 미리 모아서 DB에 쌓아 둡니다. 같은 데이터를 다시 받아도 중복이 생기지 않게 했습니다.

주식 시세

출처 토스증권 API

주기 평일 20:30 (장 마감 후)

- 문자열 가격 → 숫자, 시간 → 날짜로 변환
- (종목, 날짜) 기준 UPSERT로 다시 받아도 덮어쓰기
- 서버 시작 시 이력이 150일보다 적은 종목은 200일치 자동 보충
- 종목별로 예외 처리해서 한 종목 실패가 전체를 멈추지 않음

뉴스

출처 Google 뉴스 RSS

주기 30분마다

- 분야별·종목별 검색어로 RSS(XML) 수집
- XML 파싱 시 외부 엔티티(XXE) 차단
- 기사 링크를 UNIQUE로 두어 중복 저장 방지
- 언론사·링크·발행 시각 저장 → 화면과 AI가 같이 사용

날씨

출처 Open-Meteo

주기 10분마다 (브라우저)

- 서버를 거치지 않고 화면에서 바로 조회
- 기온·날씨·미세먼지 표시
- API 키가 필요 없는 공개 API라 프론트에서 처리

SQL과 데이터베이스 활용

테이블	용도
T_USER	회원 (이메일 · 카카오 · 네이버, 권한)
T_EMAIL_VERIFY	인증 코드 (가입 · 비밀번호 재설정 구분)
T_STOCK	종목 12개
T_STOCK_PRICE	일봉 시세 (종목+날짜 UNIQUE)
T_STOCK_DOC	종목 설명 문서 (RAG 검색 대상)
T_NEWS	뉴스 (링크 UNIQUE)
T_AI_CHAT_LOG	AI 질문·답변·도구·평가

쿼리는 회사에서 쓰던 형식(앞 콤마, WHERE 1=1)을 그대로 지켜서 작성했습니다. 오른쪽은 수익률 순위 쿼리 일부입니다.

이렇게 썼습니다

UPSERT (ON CONFLICT) — 시세·뉴스를 다시 받아도 중복 없이 반영

DISTINCT ON — 종목별 최신 시세 한 줄만 뽑아 목록 화면에 표시

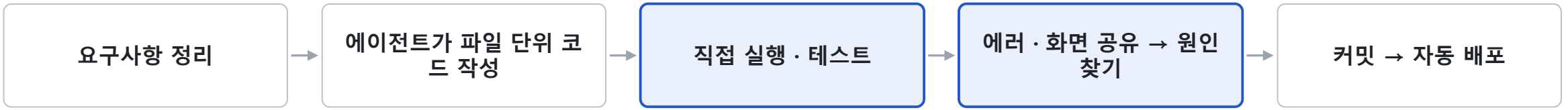
ROW_NUMBER() 윈도우 함수 — 종목별 N거래일 전 가격과 비교해 수익률 순위 계산

LEFT JOIN — 시세가 아직 없는 종목도 목록에서 빠지지 않게

```
WITH RANKED AS (
    SELECT  P.STOCK_ID
           ,P.CLOSE_PRICE
           ,ROW_NUMBER() OVER (PARTITION BY P.STOCK_ID
                               ORDER BY P.TRADE_DATE DESC) AS RN
      FROM T_STOCK_PRICE P
)
SELECT  ... ROUND((E.CLOSE_PRICE - B.CLOSE_PRICE) / B.CLOSE_PRICE * 100, 2)
```

AI 코딩 에이전트로 MVP 개발

AI 코딩 에이전트(Claude)와 같이 개발했습니다. 코드는 에이전트가 빠르게 만들고, 구조를 정하고 확인하는 일은 제가 했습니다.



제가 직접 한 것

- 구조와 규칙: 화면 단위 계층 구조, 이름 규칙, SQL 형식을 먼저 정하고 에이전트가 따르게 함
- 검증: AI 답변 숫자를 DB 값과 직접 대조, 로그로 원인 확인
- 보안: 키는 환경변수·서버 .env·GitHub Secrets에만, 노출된 키는 배포 전에 전부 재발급
- 방향 결정: 화면 구성, 기능 우선순위, 피드백 반영

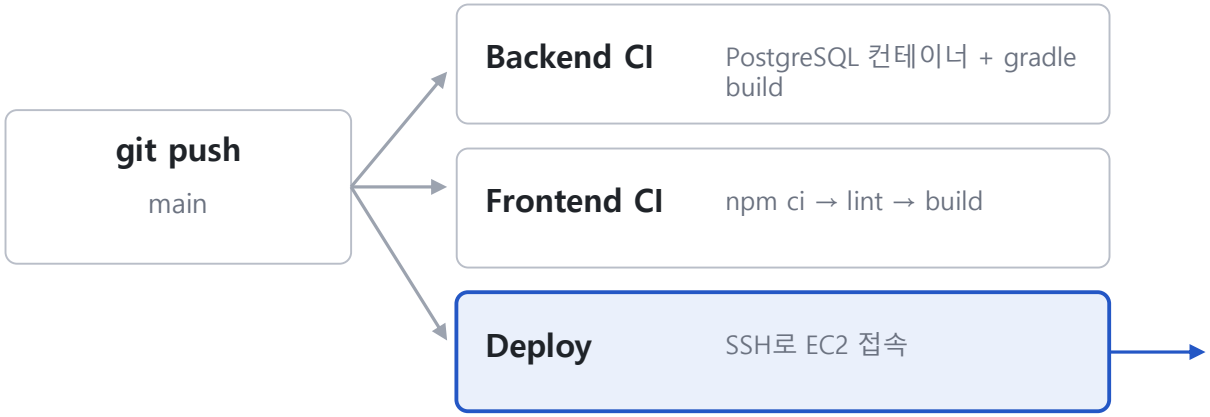
진행 순서

1일 차	회원가입 · 로그인 · 카카오/네이버 OAuth
2일 차	시세 수집 → AI 비서 → Docker → EC2 배포 → CI/CD → 피드백 개선
이후	뉴스 수집, 메인 화면, 주식 차트 화면, 관리자 화면, 계정 찾기, 종목 12개로 확대

처음 운영 서버에 올리기까지 약 2일 걸렸습니다.

CI/CD · Docker · 배포 자동화

main 브랜치에 push하면 빌드·테스트가 돌고, 동시에 EC2에 접속해 새 버전으로 다시 띄웁니다. 배포는 약 2분 걸립니다.




```
git reset --hard origin/main
docker compose up -d --build
docker image prune -f
```

EC2에서 실행되는 배포 명령

Docker 이미지

- 백엔드: JDK로 빌드 → JRE 이미지에 jar만 복사 (멀티 스테이지)
- 프론트: Node로 빌드 → nginx 이미지에 결과물만 복사
- Gradle 설정을 먼저 복사해 의존성 캐시 활용

서버 설정

- EC2 t3.micro, 스왑 2GB (빌드 중 메모리 부족 방지)
- RDS는 EC2 보안 그룹과 관리자 IP에서만 접속
- 고정 IP로 OAuth 콜백 주소 유지

비밀값 관리

- 코드에는 ${환경변수} 자리만
- 실제 값은 서버 .env(권한 600, Git 제외)와 GitHub Secrets
- 운영 서버는 SQL 파라미터 로그를 꺼서 개인정보 미기록

정리

LLM API	Claude tool use로 AI가 DB와 외부 API를 직접 조회하게 만들고, 기록을 보고 정확도를 고쳤습니다.
데이터 · SQL	시세·뉴스를 스케줄러로 모아 PostgreSQL에 쌓고, 계산은 SQL로 처리했습니다.
배포 자동화	Docker로 묶어 AWS에 올리고, push하면 자동으로 배포되게 했습니다.
AI 코딩 에이전트	에이전트로 빠르게 만들되, 구조와 검증·보안은 직접 챙겼습니다.

회사에서 해 온 연동·배치·현업 협업 경험 위에, 새 스택과 AI 활용을 직접 해 보면서 넓혀 가고 있습니다.